Lumen

Documentation complète

Votre agent IA local, privé et agentique.
macOS · Linux · Raspberry Pi · Windows

1. Vue d'ensemble

Lumen est un agent IA de bureau : une barre flottante en verre dépoli, accompagnée d'un avatar orbe animé, qui fonctionne 100 % en local et agit pour vous.

Lumen s'appuie par défaut sur **Ollama** pour faire tourner des modèles localement (LFM 2.5, Llama, Qwen, Mistral…). Aucune donnée ne quitte la machine. En option, Lumen peut utiliser les **modèles d'Apple** via l'app Raccourcis (macOS), ou une **API cloud compatible OpenAI** (OpenAI, Groq, OpenRouter…).

Les trois sources d'IA

Source	Description
Ollama local	Mode par défaut. Privé, hors-ligne, gratuit, agentique (function calling).
Raccourcis macOS	Modèles Apple, zéro configuration de modèle. Pas de function calling.
API cloud	Tout point d'accès OpenAI-compatible. La clé est stockée côté application, jamais exposée à l'interface.

Modules principaux

- **Avatar orbe** — sphère de verre WebGL (Three.js), états repos / réflexion / parole, entièrement personnalisable.
- **Heartbeat** — planificateur CRON qui déclenche un raccourci ou un workflow Fluxo à intervalle régulier.
- **Agents autonomes**, **Sandboxes** fichiers, **Fluxo** (workflows), **Mémoire cognitive**, **Base de connaissances (RAG)**.
- **Apps** : Kanban, Calendrier, Connaissances, Mindmap, Notes, Domotique, Studio d'images, Data Analysis, Éditeur de code.
- **Compagnon iPhone** — pilotage à distance via un code à usage unique (TOTP).

2. Installation & prérequis

Les scripts d'installation sont **interactifs** (chaque étape demande O/n) et **idempotents** : relançables sans risque. Ajoutez --yes pour tout installer sans question.

macOS 13+ (Apple Silicon recommandé)

Installe Ollama (via Homebrew) et télécharge le modèle par défaut. La voix et la dictée sont gérées par macOS (synthèse système ou app Raccourcis) — rien de plus à installer.

```
./scripts/install-prereqs-macos.sh
```

Linux (Debian/Ubuntu x86_64)

Installe Ollama et le modèle, puis propose les voix locales **Piper** (FR & EN) et **Kokoro**, ainsi que la dictée **whisper.cpp**.

Raspberry Pi (64-bit)

Comme Linux, en arm64. Privilégiez un petit modèle ; pour l'image et les gros modèles, déportez l'inférence sur une autre machine via Réglages → IA → URL. Le service GPIO se configure via ./scripts/install-prereqs-pi.sh.

Windows 10/11 x64

Installe Ollama (via winget) et le modèle. La voix utilise les voix système Windows ou l'app compagnon iPhone.

■ Les voix neuronales Piper / Kokoro et la dictée whisper sont disponibles sur Linux et Raspberry Pi. Sur macOS et Windows, la synthèse et la dictée passent par le système. Besoin d'un modèle cloud ? Ajoutez une clé API dans Réglages → IA, aucun script requis.

Développement

```
npm install ./dev.sh # ou : npm run tauri dev npm run tauri build # build .app / .dmg
```

3. Guide rapide

- Ouvrez la barre, tapez votre question, validez avec Entrée ou le bouton →.
- Le bouton ■ ouvre les Paramètres ; le bouton ■ ouvre la Boîte à outils (tous les modules).
- Le sélecteur de **type de chat** propose : **Chat** (conversation simple) et **Function Calling** (l'IA peut appeler des outils : workflows Fluxo, domotique, mémoire, rappels, recherche documents).
- Le microphone lance la dictée ; le haut-parleur (auto-lecture) énonce les réponses.
- Les boutons d'outils s'adaptent à la taille de barre choisie (petit / moyen / grand) ; les moins utilisés se replient dans un menu « Plus ».

■ La synthèse vocale du chat réutilise le canal du Heartbeat. Sur macOS la commande système « say » est utilisée ; le verrou de lecture est relâché avant la lecture pour qu'un appel figé ne bloque jamais les énonciations suivantes.

4. IA & modèles (configuration)

La source d'IA se choisit dans Réglages → Général → Source d'IA. Selon la source, des champs supplémentaires apparaissent.

Ollama local

Réglage	Détail
URL d'Ollama	http://localhost:11434 par défaut.
Modèle	Sélection parmi les modèles installés (ex. liquidai/lfm2.5-350m:latest).
Mode raisonnement	Laisse le modèle « réfléchir » (modèles compatibles : qwen3, deepseek-r1).
Taille du contexte	Curseur à paliers : Défaut du modèle, 2k, 4k, 8k, 16k, 32k, 64k, 128k tokens.
Keep-alive	Durée de maintien en mémoire (ex. 5m, 1h, -1 = toujours, 0 = décharger).

API cloud (OpenAI-compatible)

Renseignez la clé API, l'URL de base (ex. https://api.openai.com/v1) et le modèle (ex. gpt-4o-mini). La clé est conservée dans un fichier protégé côté application et n'est jamais exposée à l'interface. Laissez le champ vide pour conserver la clé enregistrée.

■ Le backend cloud bascule tout le pipeline LLM (barre, outils, agents, blocs Fluxo) sur le point d'accès OpenAI-compatible choisi.

5. Agents autonomes

Un agent = un nom, une **sandbox** attribuée, des **skills** (workflows Fluxo enchaînés exposés comme outils), une **todo list**, et une planification **CRON**. Les agents reposent sur le function calling d'Ollama — indisponibles en source Apple.

Le cycle de travail (un tick)

- L'agent prend sa prochaine tâche échue (date du jour ou passée, ou sans date).
- Il appelle ses skills comme des outils pour accomplir la tâche, dans sa sandbox.
- Il produit un compte rendu écrit dans sa sandbox et poste un message dans la Messagerie.

Tâches programmées (dates)

Une tâche sans date est disponible immédiatement. Une tâche avec une date n'est visible par l'agent qu'à partir de ce jour (badge ■ tant que l'échéance est future, badge vert le jour venu). Les tâches datées des agents apparaissent aussi dans l'app Calendrier.

■ Supprimer un agent retire sa configuration mais ne touche pas à sa sandbox : les fichiers produits restent dans l'explorateur de sandboxes.

6. Sandboxes fichiers

Une sandbox est un espace fichiers **isolé et limité en taille** où un agent peut créer et analyser des fichiers. Toutes les opérations sont cantonnées à la racine de la sandbox (anti-traversée de chemin, quota d'espace, versioning).

C'est une sandbox **applicative**, pas une isolation noyau : elle protège contre un agent qui dérape ou un fichier piégé, à condition de ne pas exposer d'exécution de commandes ni de réseau arbitraire.

Accès depuis les workflows

Les blocs read_file et write_file de Fluxo savent viser une sandbox nommée (via le pont Lumen, qui conserve quota et versioning) ou se confiner directement au dossier de l'agent.

7. Fluxo — moteur d'automatisation

Fluxo enchaîne des **blocs** (actions) en workflows, conçus dans le Designer en glisser-déposer. Un modèle compatible function calling peut déclencher lui-même vos workflows. Le moteur communique avec Lumen via un serveur loopback local (http://127.0.0.1:8765).

Quelques blocs Lumen

Bloc	Rôle
Notification Lumen	Pousse une notification (bulle flottante + centre de notifications).
Widget Lumen	Affiche un widget structuré (gabarit + titre + données JSON) dans la discussion.
Générer une image	Bonsai local ou OpenAI ; enregistre le fichier dans une sandbox.
Consolider la mémoire	Digère les souvenirs épisodiques récents en connaissances durables.
Rechercher dans les connaissances	Recherche RAG ; renvoie les extraits pertinents pour la suite du workflow.
Lancer un agent	Déclenche un agent (chaîne de skills) — idéal depuis le Heartbeat.

8. Calendrier & rappels

L'app Calendrier affiche une grille mensuelle agrégeant vos **rappels datés** et les **tâches programmées des agents** (en lecture seule, badge agent).

Rappels qui se déclenchent

Un rappel a un titre, une note, une échéance horodatée et une récurrence (aucune, quotidienne, hebdomadaire, mensuelle). Un ticker d'arrière-plan vérifie les échéances toutes les 30 secondes et déclenche une notification (cloche + pulsation de l'orbe) à terme. Les rappels manqués hors-ligne sont rattrapés ; les récurrents se ré-arment.

Création par l'IA et les agents

- Assistant de la barre : l'outil create_reminder crée un rappel daté à partir du langage naturel (« rappelle-moi demain 9h »).
- Agents / blocs Fluxo : la route loopback POST /reminders permet de poser un rappel daté.

9. Base de connaissances (RAG)

L'app Connaissances importe des documents (PDF, texte, Markdown, CSV, JSON, code) que Lumen découpe en extraits et vectorise **localement**, via le backend IA actif. L'assistant et les agents peuvent alors s'appuyer dessus.

Architecture

Base vectorielle **SQLite + sqlite-vec** (fichier rag.db), portable sur toutes les plateformes, sans dépendance système. Les embeddings sont calculés par Ollama (nomic-embed-text par défaut) ou l'API cloud (text-embedding-3-small).

Recherche hybride

La recherche combine deux approches pour de meilleurs résultats :

- **Mot-clé (FTS5/BM25)** — prioritaire, idéale pour les noms propres et termes exacts.
- **Sémantique (cosinus)** — en complément, avec un seuil de pertinence pour écarter le hors-sujet.

Réglages d'embedding simplifiés

Découpage	Usage
Petit (~500 car.)	Réponses précises : Q/R, fiches, annuaires.
Moyen (~1000 car.)	Équilibré (défaut).
Grand (~1800 car.)	Conserve le contexte : prose, rapports.

■ Changer de découpage ou de modèle d'embedding s'applique aux prochains imports. Le bouton Réindexer reconstruit l'index à partir du texte déjà stocké, sans réimport.

Accès IA

L'assistant utilise l'outil search_documents ; les workflows utilisent le bloc « Rechercher dans les connaissances » ; les agents passent par POST /rag/search.

10. Lumen OS & mémoire

Lumen OS est le bureau immersif : fenêtres, widgets, menu Démarrer, et l'assistant intégré. Les apps s'y ouvrent en fenêtres. Le thème (clair/sombre) et la couleur d'accent sont partagés avec le reste de Lumen.

Mémoire cognitive

La mémoire de Lumen comporte **8 types** (dont épisodique et prospective) avec une politique d'accès cloisonnée : un contexte (barre, agent, bloc) ne lit/écrit que les types autorisés. Le rappel est pondéré (pertinence + récence + saillance). L'app Mémoire permet de consulter, épingler, modifier et oublier des souvenirs.

Profil utilisateur

Le profil (prénom, façon de s'adresser, style de réponse, centres d'intérêt, objectifs) est stocké localement et accessible à l'assistant pour personnaliser ses réponses.

11. Domotique & compagnon iPhone

Domotique

L'app Domotique gère des objets connectés et leurs firmwares (ESP32 / Pico, MicroPython), via un protocole local léger. Les agents et workflows peuvent lire l'état d'un objet et envoyer des commandes sécurisées en local.

Compagnon iPhone

Depuis Safari sur l'iPhone (même réseau Wi-Fi ou VPN), ouvrez l'adresse affichée dans Réglages → Compagnon pour piloter Lumen à distance. L'accès est protégé par un code à usage unique (TOTP) ; une session est ensuite mémorisée sur le téléphone. La voix (énoncer / dictée) est gérée par l'iPhone.

12. API développeur (loopback)

Lumen expose un serveur local sur http://127.0.0.1:8765 (loopback uniquement, jamais le réseau). Quelques routes utiles aux raccourcis, agents et blocs Fluxo :

Route	Description
GET/PUT/POST/DELETE /mem/<clé>	Mémoire partagée (namespaces todo/ state/ user/ store/).
POST /notify	Notification flottante.
POST /agent/run	Déclenche un agent.
POST /agent/task	Ajoute une tâche à la todo d'un agent.
GET/POST /reminders	Liste / crée un rappel daté.
POST /rag/search	Recherche dans la base de connaissances.
POST /run/progress	Callback de progression d'un run (mode Action).

■ Ces routes ne sont servies que sur le loopback : inaccessibles hors de la machine par construction. L'accès web pour l'IA reste volontairement sous contrôle de l'utilisateur (via Fluxo et skills) pour éviter les risques d'injection.

— Fin de la documentation Lumen —